

Marathi News Intelligence Pipeline

Source code

1. Aim

Mini Project based on Real Life NLP Application.

2. Objectives

- To understand natural language processing and to learn how to apply basic algorithms in this field.
- To design and implement applications based on natural language processing.

Outcomes: Be able to apply NLP techniques to design real world NLP applications such as machine translation, text categorization, text summarization, information extraction, etc.

3. Hardware / Software Required

LANGUAGE	Python 3.10 or newer; JavaScript for the in-browser demo engine.
LIBRARIES	PyTorch, Hugging Face Transformers, NumPy, pandas, scikit-learn, Streamlit, BeautifulSoup, pytest.
MODELS	MahaBERT topic / NER / sentiment / sentence-similarity models (L3Cube, Pune) and NLLB-200 distilled 600M (Meta AI).
DATASETS	XL-Sum Marathi (13,627 BBC Marathi articles with summaries), MahaNews (11,721 labelled headlines), and live RSS feeds.
HARDWARE	Any laptop. A GPU (NVIDIA CUDA or Apple Silicon) only makes it faster; developed on a 4 GB RTX 3050 and an Apple Silicon Mac.

4. Theory

Natural Language Processing (NLP) is the branch of artificial intelligence that lets computers work with human language. This project applies seven standard NLP tasks to Marathi news, a language with far fewer ready-made tools than English.

4.1 Morphological analysis (task I)

Marathi is **agglutinative**: grammar is added by joining suffixes to a root, so what English writes as several words becomes one word.

घर	house	(root)
घरात	= घर + आत	in the house
घरातून	= घर + आतून	from inside the house
घरांमधून	= घर + ां + मधून	from inside the houses
घराच्या	= घर + आ + च्या	of the house

A whitespace tokeniser treats these as five unrelated words, which harms every component that compares words. A **morphological analyser** reduces each form to its root. This one is **rule-based**: it strips suffixes in four ordered layers (clitic, postposition, verb ending, case and number), then repairs the stem. Length guards count **aksharas** (written syllables), not Unicode code points, because Devanagari vowel signs are separate code points. In the 3,000-article corpus, 36% of tokens carry a removable suffix, and normalising them shrinks the index vocabulary from 186,593 to 84,480 types (54.7% smaller).

4.2 Information retrieval (task C)

BM25 ranks a document higher when it contains the query terms often, when those terms are rare in the collection, and it penalises length. **Dense retrieval** embeds text as vectors with a neural encoder and ranks by cosine similarity, matching paraphrases that share no words. **Reciprocal Rank Fusion** merges the two ranked lists by rank rather than score, because the two scores are on different scales. Quality is reported as recall@k and **MRR**.

4.3 Text summarisation (task E)

TextRank is extractive and graph-based: sentences are nodes, edges are TF-IDF cosine similarity, and **PageRank** scores how central each sentence is. **Maximal Marginal Relevance** then picks the final sentences, trading relevance against redundancy. Summaries are scored with **ROUGE**, the word overlap with human-written summaries.

4.4 Categorisation, extraction and sentiment (tasks D, G, F)

These use **BERT**-family transformers pre-trained on Marathi (MahaBERT, L3Cube Pune). Categorisation predicts one of 12 topics; **named entity recognition** is token classification (each token labelled as part of a person, location, organisation, date...); sentiment classifies polarity per sentence, aggregated into a document tone. **Fine-tuning** a pre-trained model on the task's own data raised topic accuracy from 0.847 to 0.954.

4.5 Translation and distillation (task A)

Translation uses **NLLB-200**, a multilingual sequence-to-sequence transformer, sentence by sentence. For the browser demo, **knowledge distillation** was used: small linear **student** models were trained on the large models' own output over 12,262 articles, and agree with them on 83.0% of topics and 75.7% of sentence sentiment labels (F1 0.72 on entities) while fitting in a few megabytes.

4.6 System design

The seven tasks form one pipeline. The analyser runs first because retrieval, summarisation and entity counting all consume its roots; the enriched articles are then indexed for search. Every claim is measured against a simple baseline, and negative results are reported too.

5. Source code

This listing shows the core of each task. The complete project — 5,830 lines across 28 files, including the data-preparation scripts, the Streamlit dashboard, the demo page's JavaScript engine and all 59 tests — is at github.com/anishc23/nlp-project. Long docstrings are shortened to one line, and `# ...` marks code left out between the parts shown.

I — Morphological analyser (written from scratch)

src/morph.py

7 of its definitions

Strips the suffix chain layer by layer, counting aksharas rather than characters, and returns the root plus the

grammatical features removed. Everything else in the project consumes these roots.

```
# ...
def akshara_len(s: str) -> int:
    """Number of base characters, ignoring combining vowel signs and virama."""
    return sum(1 for ch in s if ch not in _MARKS)
# ...
def _restore_stem(stem_form: str) -> str:
    """Normalise the stem left behind after suffix removal. ..."""
    if not stem_form:
        return stem_form

    # A dangling virama is never a valid word end (पुण्य- from पुण्यात्).
    while stem_form.endswith(VIRAMA):
        stem_form = stem_form[:-1]

    if stem_form and stem_form[-1] in _FINAL_MATRAS and akshara_len(stem_form) >= MIN_STEM:
        stem_form = stem_form[:-1]

    return stem_form
# ...
def _strip_once(word: str, layers) -> Optional[Tuple[str, str, str]]:
    """Remove at most one suffix. Returns (stem, suffix, feature) or None."""
    for layer_name, table in layers:
        floor = MIN_DERIV_STEM if layer_name == "deriv" else MIN_STEM
        for suf, feat, repl in table:
            if not word.endswith(suf):
                continue
            candidate = word[:-len(suf)] + repl
            if akshara_len(candidate) >= floor:
                return candidate, suf, feat
    return None
# ...
@lru_cache(maxsize=200_000)
def analyse(word: str, derivational: bool = False) -> Analysis:
    """Analyse one token into root + morphological features."""
    surface = word
    word = unicodedata.normalize("NFC", word)

    if surface in EXCEPTIONS:
        return Analysis(surface=surface, root=EXCEPTIONS[surface])

    layers = _LAYERS + [_DERIV_LAYER] if derivational else _LAYERS
    feats: List[str] = []
    strips: List[str] = []

    for _ in range(MAX_STRIPS):
        hit = _strip_once(word, layers)
        if hit is None:
            break
        new_stem, suf, feat = hit
        word = new_stem
        feats.append(feat)
        strips.append(suf)

    root = _restore_stem(word)
    return Analysis(surface=surface, root=root, features=feats, stripped=strips)
# ...
def stem(word: str) -> str:
    """Convenience wrapper: surface form -> normalised root."""
    return analyse(word).root
# ...
def tokenize(text: str) -> List[str]:
    """Script-aware word tokeniser (Devanagari, Latin, digits)."""
    return _TOKEN_RE.findall(text or "")
# ...
def normalize_tokens(text: str, *, use_morph: bool = True,
                    drop_stopwords: bool = True) -> List[str]:
    """Full normalisation path used for indexing and similarity. ..."""
    out = []
    for tok in tokenize(text):
```

```
if drop_stopwords and tok in STOPWORDS:
    continue
t = stem(tok) if use_morph else tok
if drop_stopwords and t in STOPWORDS:
    continue
if akshara_len(t) >= MIN_STEM or t.isascii():
    out.append(t)
return out
# ...
```

C — Information retrieval

[src/retrieve.py](#)

3 of its definitions

BM25 implemented directly so the tokenisation path can be switched between raw words and analyser roots — that

switch is the project's central experiment. Dense retrieval and rank fusion complete the hybrid retriever.

```
# ...
@dataclass
class BM25Index:
    use_morph: bool = True
    postings: Dict[str, List[tuple]] = field(default_factory=lambda: defaultdict(list))
    doc_len: List[int] = field(default_factory=list)
    doc_ids: List[int] = field(default_factory=list)
    avg_len: float = 0.0

    def build(self, docs: List[str], doc_ids: Optional[List[int]] = None):
        self.postings = defaultdict(list)
        self.doc_len = []
        self.doc_ids = doc_ids if doc_ids is not None else list(range(len(docs)))

        for i, text in enumerate(docs):
            toks = normalize_tokens(text, use_morph=self.use_morph)
            self.doc_len.append(len(toks))
            for term, tf in Counter(toks).items():
                self.postings[term].append((i, tf))

        self.avg_len = (sum(self.doc_len) / len(self.doc_len)) if self.doc_len else 0.0
        self.postings = dict(self.postings)
        return self

    def search(self, query: str, top_k: int = 10) -> List[Hit]:
        n = len(self.doc_len)
        if n == 0:
            return []
        q_terms = normalize_tokens(query, use_morph=self.use_morph)
        scores = np.zeros(n, dtype="float32")

        for term in q_terms:
            posting = self.postings.get(term)
            if not posting:
                continue
            df = len(posting)
            # BM25 idf with the +0.5 smoothing; clamped at 0 so that terms
            # appearing in more than half the corpus cannot subtract score.
            idf = max(math.log((n - df + 0.5) / (df + 0.5) + 1.0), 0.0)
            for doc_i, tf in posting:
                denom = tf + K1 * (1 - B + B * self.doc_len[doc_i] / max(self.avg_len, 1e-9))
                scores[doc_i] += idf * (tf * (K1 + 1)) / max(denom, 1e-9)

        top = np.argsort(-scores)[:top_k]
        return [Hit(doc_id=self.doc_ids[i], score=float(scores[i]), rank=r, source="bm25")
                for r, i in enumerate(top) if scores[i] > 0]

    def save(self, path: Path):
        Path(path).write_bytes(pickle.dumps(self))

    @staticmethod
    def load(path: Path) -> "BM25Index":
        return pickle.loads(Path(path).read_bytes())

# ...
def reciprocal_rank_fusion(runs: List[List[Hit]], top_k: int = 10) -> List[Hit]:
    """Combine ranked lists by summing 1/(k + rank)."""
    fused: Dict[int, float] = defaultdict(float)
    for run in runs:
        for hit in run:
            fused[hit.doc_id] += 1.0 / (RRF_K + hit.rank + 1)
    ordered = sorted(fused.items(), key=lambda kv: -kv[1])[:top_k]
    return [Hit(doc_id=d, score=s, rank=r, source="hybrid")
            for r, (d, s) in enumerate(ordered)]

# ...
class HybridRetriever:
    """BM25 + dense with RRF, with either half optional."""

    def __init__(self, bm25: BM25Index = None, dense: DenseIndex = None):
        self.bm25 = bm25
```

```

        self.dense = dense

    def search(self, query: str, top_k: int = 10,
               mode: str = "hybrid") -> List[Hit]:
        if mode == "bm25":
            return self.bm25.search(query, top_k)
        if mode == "dense":
            return self.dense.search(query, top_k)
        runs = []
        if self.bm25 is not None:
            runs.append(self.bm25.search(query, top_k * 3))
        if self.dense is not None:
            runs.append(self.dense.search(query, top_k * 3))
        if not runs:
            return []
        return reciprocal_rank_fusion(runs, top_k)

    @staticmethod
    def load_default() -> "HybridRetriever":
        bm25 = BM25Index.load(config.BM25_INDEX) if config.BM25_INDEX.exists() else None
        dense = DenseIndex.load(config.DENSE_INDEX) if config.DENSE_INDEX.exists() else None
        return HybridRetriever(bm25=bm25, dense=dense)

# ...

```

E — Text summarisation

src/summarize.py

5 of its definitions

TextRank: a sentence-similarity graph built from morphologically normalised TF-IDF, ranked by PageRank, then

de-duplicated with MMR.

```
# ...
def _tfidf_matrix(sentences: List[str]) -> np.ndarray:
    """Morphologically normalised TF-IDF, built without sklearn's tokenizer."""
    docs = [normalize_tokens(s) for s in sentences]
    vocab = {}
    for d in docs:
        for t in d:
            vocab.setdefault(t, len(vocab))
    if not vocab:
        return np.zeros((len(sentences), 1), dtype="float64")

    # float64 so the page's JavaScript port (always float64) ranks identically.
    tf = np.zeros((len(docs), len(vocab)), dtype="float64")
    for i, d in enumerate(docs):
        for t in d:
            tf[i, vocab[t]] += 1.0
    tf /= np.maximum(tf.sum(1, keepdims=True), 1e-9)

    df = (tf > 0).sum(0)
    idf = np.log((1 + len(docs)) / (1 + df)) + 1.0
    return tf * idf

# ...
def _cosine_graph(vectors: np.ndarray) -> np.ndarray:
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)
    unit = vectors / np.maximum(norms, 1e-9)
    sim = unit @ unit.T
    np.fill_diagonal(sim, 0.0)
    return np.clip(sim, 0.0, None)

# ...
def _pagerank(sim: np.ndarray) -> np.ndarray:
    """Power iteration; falls back to uniform scores on a disconnected graph."""
    n = sim.shape[0]
    if n == 0:
        return np.zeros(0)
    row = sim.sum(1, keepdims=True)
    # Dangling rows get a uniform transition so probability mass is conserved.
    transition = np.where(row > 0, sim / np.maximum(row, 1e-9), 1.0 / n)
    scores = np.full(n, 1.0 / n)
    for _ in range(MAX_ITER):
        nxt = (1 - DAMPING) / n + DAMPING * (transition.T @ scores)
        if np.abs(nxt - scores).sum() < TOL:
            return nxt
        scores = nxt
    return scores

# ...
def _mmr(order: List[int], sim: np.ndarray, k: int,
         lambda_: float = 0.7) -> List[int]:
    """Greedy pick k sentences trading relevance against redundancy. ..."""
    n = max(len(order), 1)
    relevance = {idx: 1.0 - pos / n for pos, idx in enumerate(order)}

    selected: List[int] = []
    candidates = list(order)
    while candidates and len(selected) < k:
        best, best_score = None, -np.inf
        for idx in candidates:
            redundancy = max((sim[idx, j] for j in selected), default=0.0)
            score = round(lambda_ * relevance[idx] - (1 - lambda_) * redundancy, 12)
            if score > best_score:
                best, best_score = idx, score
        selected.append(best)
        candidates.remove(best)
    return selected

# ...
def summarize(text: str, k: int = 3, method: str = "textrank",
             lambda_: float = 0.7) -> Summary:
    """Return the k most representative sentences, in original order."""
    sentences = [s for s in sent_tokenize(text) if len(s.split()) >= 3]
    if not sentences:
```

```

    return Summary(sentences=[], indices=[], method=method)
if len(sentences) <= k:
    return Summary(sentences=sentences, indices=list(range(len(sentences))),
                    method=method)

if method == "lead":
    idx = list(range(k))
    return Summary(sentences=[sentences[i] for i in idx], indices=idx,
                    method=method)

if method == "embedding":
    from src import models
    vectors = models.embed(sentences)
else:
    vectors = _tfidf_matrix(sentences)

sim = _cosine_graph(vectors)
# Scores are rounded before a stable sort so that exact ties (repeated
# sentences are common in news) keep document order. Without the rounding
# the order of a tie depends on last-bit floating-point noise, which
# differs between numpy and the page's JavaScript port.
ranked = [int(i) for i in np.argsort(-np.round(_pagerank(sim), 12), kind="stable")]
chosen = sorted(_mmr(ranked, sim, k, lambda_))
return Summary(sentences=[sentences[i] for i in chosen], indices=chosen,
                method=method)

# ...

```

D — Text categorisation

src/categorize.py

2 of its definitions

The 12-topic MahaBERT classifier. `active_model()` documents why the better-scoring fine-tuned head is

deliberately not the default.

```
# ...
def active_model() -> str:
    """The model the pipeline uses by default. ..."""
    choice = os.environ.get("MNIP_TOPIC_MODEL", "").lower()
    if choice in ("custom", "finetuned", "local"):
        if (config.CUSTOM_TOPIC_DIR / "config.json").exists():
            return str(config.CUSTOM_TOPIC_DIR)
        raise FileNotFoundError(
            f"MNIP_TOPIC_MODEL={choice} but no checkpoint at "
            f"{config.CUSTOM_TOPIC_DIR}. Run scripts/train_classifier.py first.")
    return config.MODEL_TOPIC

# ...
@torch.no_grad()
def classify(texts: List[str], model_name: str = None,
            batch_size: int = None) -> List[TopicPrediction]:
    """Classify a batch of documents into news topics."""
    if isinstance(texts, str):
        texts = [texts]
    model_name = model_name or active_model()
    tok, mod = models.load_classifier(model_name)
    id2label = mod.config.id2label
    bs = batch_size or config.BATCH_SIZE
    results: List[TopicPrediction] = []

    for i in range(0, len(texts), bs):
        chunk = [t if t else " " for t in texts[i:i + bs]]
        enc = tok(chunk, padding=True, truncation=True,
                  max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
        probs = mod(**enc).logits.float().softmax(-1).cpu()
        for row in probs:
            idx = int(row.argmax())
            results.append(TopicPrediction(
                label=id2label[idx],
                score=float(row[idx]),
                all_scores={id2label[j]: float(row[j]) for j in range(len(row))},
            ))
    return results

# ...
```

G — Information extraction

[src/ner.py](#)

4 of its definitions

MahaNER emits flat labels, so spans are merged here, snapped back to whole words (wordpiece tokenisers split

Devanagari mid-letter), and grouped by de-inflected root.

```
# ...
@dataclass
class Entity:
    text: str
    label: str
    start: int
    end: int
    score: float

    @property
    def canonical(self) -> str:
        """Case-stripped form used to aggregate mentions across the corpus. ..."""
        parts = self.text.split()
        if not parts:
            return self.text
        return " ".join(parts[:-1] + [morph_stem(parts[-1])]).strip()

    def __str__(self) -> str:
        return f"{self.text} ({self.label}, {self.score:.2f})"

# ...
def _snap_to_word(text: str, start: int, end: int) -> tuple:
    """Widen a span to whole-word boundaries. ..."""
    while start > 0 and text[start - 1] not in _BOUNDARY:
        start -= 1
    while end < len(text) and text[end] not in _BOUNDARY:
        end += 1
    return start, end

# ...
@torch.no_grad()
def extract(text: str, max_length: int = None) -> List[Entity]:
    """Extract entities from a single document."""
    if not text or not text.strip():
        return []

    tok, mod = models.load_token_classifier(config.MODEL_NER)
    id2label = mod.config.id2label
    max_length = max_length or config.MAX_LEN

    enc = tok(text, return_offsets_mapping=True, truncation=True,
              max_length=max_length, return_tensors="pt")
    offsets = enc.pop("offset_mapping")[0].tolist()
    enc.pop("special_tokens_mask", None)
    enc = {k: v.to(config.DEVICE) for k, v in enc.items()}

    probs = mod(**enc).logits.float().softmax(-1)[0].cpu()
    label_ids = probs.argmax(-1).tolist()
    scores = probs.max(-1).values.tolist()

    spans: List[Entity] = []
    cur = None
    for (start, end), lid, sc in zip(offsets, label_ids, scores):
        if start == end:
            # special token
            continue
        label = id2label[lid]
        if label == "Other":
            cur = None
            continue
        # Continue the current span when the label matches and the gap is only
        # whitespace; otherwise start a new one.
        if cur and cur.label == label and start - cur.end <= 1:
            cur.end = end
            cur.score = (cur.score + sc) / 2
        else:
            cur = Entity(text="", label=label, start=start, end=end, score=sc)
            spans.append(cur)

    for sp in spans:
        sp.start, sp.end = _snap_to_word(text, sp.start, sp.end)
        sp.text = text[sp.start:sp.end].strip()
```

```

# Snapping can make two adjacent spans of the same type identical; collapse
# runs of duplicates while preserving order.
deduped: List[Entity] = []
for sp in spans:
    if not sp.text:
        continue
    if deduped and deduped[-1].text == sp.text and deduped[-1].label == sp.label:
        continue
    deduped.append(sp)
return deduped

# ...
def summarize_entities(entities: List[Entity]) -> Dict[str, List[str]]:
    """Group entities by type, de-duplicated and ordered by frequency. ..."""
    counts: Dict[tuple, int] = Counter()
    surfaces: Dict[tuple, Counter] = defaultdict(Counter)
    for e in entities:
        key = (e.label, e.canonical)
        counts[key] += 1
        surfaces[key][e.text] += 1

    buckets: Dict[str, List[str]] = defaultdict(list)
    for (label, _canon), _n in counts.most_common():
        buckets[label].append(surfaces[(label, _canon)].most_common(1)[0][0])
    return dict(buckets)

# ...

```

F — Sentiment analysis

src/sentiment.py

3 of its definitions

Scored per sentence, then aggregated into a document tone by length-weighted expected polarity, and again per

entity.

```
# ...
@torch.no_grad()
def classify_sentences(sentences: List[str],
                      batch_size: int = None) -> List[SentenceSentiment]:
    """Polarity of each sentence."""
    if not sentences:
        return []
    tok, mod = models.load_classifier(config.MODEL_SENTIMENT)
    id2label = mod.config.id2label
    bs = batch_size or config.BATCH_SIZE
    out: List[SentenceSentiment] = []

    for i in range(0, len(sentences), bs):
        chunk = [s if s else " " for s in sentences[i:i + bs]]
        enc = tok(chunk, padding=True, truncation=True,
                  max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
        probs = mod(**enc).logits.float().softmax(-1).cpu()
        for text, row in zip(chunk, probs):
            idx = int(row.argmax())
            label = id2label[idx]
            # Expected polarity rather than the argmax class: keeps a
            # 0.51/0.49 Negative/Neutral sentence from counting as strongly
            # negative when averaged over a document.
            pol = sum(POLARITY[id2label[j]] * float(row[j]) for j in range(len(row)))
            out.append(SentenceSentiment(text=text, label=label,
                                         score=float(row[idx]), polarity=pol))

    return out

# ...
def analyse_document(text: str) -> DocSentiment:
    """Length-weighted document tone built from sentence-level polarity."""
    sents = sent_tokenize(text)
    if not sents:
        return DocSentiment(label="Neutral", polarity=0.0)
    scored = classify_sentences(sents)
    weights = [max(len(s.text), 1) for s in scored]
    total = sum(weights)
    pol = sum(s.polarity * w for s, w in zip(scored, weights)) / total
    return DocSentiment(label=_label_from_polarity(pol), polarity=pol,
                        sentences=scored)

# ...
def entity_sentiment(doc: DocSentiment, entities) -> Dict[str, float]:
    """Mean polarity of the sentences each entity is mentioned in. ..."""
    scores: Dict[str, List[float]] = defaultdict(list)
    for sent in doc.sentences:
        for ent in entities:
            if ent.text and ent.text in sent.text:
                scores[ent.text].append(sent.polarity)
    return {k: sum(v) / len(v) for k, v in scores.items() if v}

# ...
```

A — Machine translation

src/translate.py

3 of its definitions

NLLB-200, applied sentence by sentence so long articles stay within the model's reliable length.

```
# ...
def _target_lang_id(tok) -> int:
    """Resolve the forced BOS token for the target language. ..."""
    code = config.NLLB_TGT_LANG
    mapping = getattr(tok, "lang_code_to_id", None)
    if isinstance(mapping, dict) and code in mapping:
        return mapping[code]
    tid = tok.convert_tokens_to_ids(code)
    if tid is None or tid == tok.unk_token_id:
        raise ValueError(f"Could not resolve NLLB language token {code!r}")
    return tid
# ...
@torch.no_grad()
def translate(texts, max_new_tokens: int = 256,
              batch_size: int = None, num_beams: int = 4) -> List[str]:
    """Translate Marathi strings to English."""
    single = isinstance(texts, str)
    if single:
        texts = [texts]
    texts = [t if t and t.strip() else " " for t in texts]

    tok, mod = models.load_translator()
    bos = _target_lang_id(tok)
    bs = batch_size or max(1, (config.BATCH_SIZE // 4))
    out: List[str] = []

    for i in range(0, len(texts), bs):
        chunk = texts[i:i + bs]
        enc = tok(chunk, padding=True, truncation=True, max_length=512,
                  return_tensors="pt").to(config.DEVICE)
        gen = mod.generate(**enc, forced_bos_token_id=bos,
                           max_new_tokens=max_new_tokens, num_beams=num_beams)
        out.extend(tok.batch_decode(gen, skip_special_tokens=True))

    return out[0] if single else out
# ...
def translate_document(text: str, max_sentences: int = 12) -> str:
    """Sentence-split a long document, translate, and re-join."""
    sents = sent_tokenize(text)[:max_sentences]
    if not sents:
        return ""
    return " ".join(translate(sents))
# ...
```

The pipeline and its evaluation

src/pipeline.py

1 of its definitions

Runs every stage over one article and returns a single enriched record.

```
# ...
def process(title: str, text: str, *, url: str = "", doc_id: str = None,
            summary_k: int = 3, summary_method: str = "textrank",
            translate: bool = False) -> Document:
    """Run every module over a single article."""
    doc = Document(id=doc_id, title=title or "", text=text or "", url=url)

    # --- D: topic -----
    clf_input = f"{doc.title}. {doc.text}" if doc.title else doc.text
    pred = categorize.classify([clf_input])[0]
    doc.topic, doc.topic_score, doc.topic_scores = pred.label, pred.score, pred.all_scores

    # --- E: summary -----
    summ = summarize.summarize(doc.text, k=summary_k, method=summary_method)
    doc.summary, doc.summary_sentences = summ.text, summ.sentences

    # --- G: entities -----
    # NER is run over title + summary rather than the full article: the model
    # truncates at 256 tokens anyway, and the summary is a denser carrier of
    # the article's principal entities than an arbitrary 256-token prefix.
    ent_source = f"{doc.title}. {doc.summary}".strip()
    ents = ner.extract(ent_source)
    doc.entities = ner.summarize_entities(ents)

    # --- F: sentiment -----
    sent_doc = sentiment.analyse_document(doc.summary or doc.text)
    doc.sentiment = sent_doc.label
    doc.polarity = sent_doc.polarity
    doc.sentiment_distribution = sent_doc.distribution
    doc.entity_sentiment = sentiment.entity_sentiment(sent_doc, ents)

    # --- A: translation (optional) -----
    if translate:
        from src import translate as mt
        doc.title_en = mt.translate(doc.title) if doc.title else ""
        doc.summary_en = mt.translate_document(doc.summary) if doc.summary else ""

    # --- I: morphology statistics -----
    doc.tokens, doc.inflected_ratio = _morph_stats(doc.text)
    return doc
# ...
```

The morphology ablation: the same queries against the same corpus, indexed with and without the analyser.

```
# ...
def rouge_n(pred: List[str], gold: List[str], n: int) -> float:
    p, g = _ngrams(pred, n), _ngrams(gold, n)
    overlap = sum((p & g).values())
    return _f1(overlap, max(sum(p.values()), 0), max(sum(g.values()), 0))

# ...
def eval_retrieval(corpus, n_queries=300, top_k=10, include_dense=True):
    """Known-item retrieval: each query is one document's title, and the correct ..."""
    from src.retrieve import BM25Index, DenseIndex, HybridRetriever

    random.seed(SEED)
    bodies = [r.get("text", "") for r in corpus]
    doc_ids = list(range(len(corpus)))

    eligible = [i for i, r in enumerate(corpus)
                 if len(r.get("title", "").split()) >= 4 and len(r.get("text", "")) > 300]
    queries = random.sample(eligible, min(n_queries, len(eligible)))
    print(f" {len(corpus)} docs indexed, {len(queries)} known-item queries\n")

    runs = {}
    built = {}

    for use_morph in (False, True):
        tag = "BM25 + morphology" if use_morph else "BM25 (plain tokens)"
        t0 = time.time()
        idx = BM25Index(use_morph=use_morph).build(bodies, doc_ids)
        built[use_morph] = idx
        results = []
        for qi in queries:
            hits = idx.search(corpus[qi]["title"], top_k=top_k)
            rank = next((h.rank for h in hits if h.doc_id == qi), None)
            results.append(rank)
        runs[tag] = results
        runs[tag + "::vocab"] = len(idx.postings)
        runs[tag + "::time"] = time.time() - t0

    if include_dense:
        t0 = time.time()
        dense_texts = [f"{r.get('summary', '')} or {r.get('text', '')[:600]}" for r in corpus]
        dense = DenseIndex().build(dense_texts, doc_ids, show_progress=False)
        results = []
        for qi in queries:
            hits = dense.search(corpus[qi]["title"], top_k=top_k)
            rank = next((h.rank for h in hits if h.doc_id == qi), None)
            results.append(rank)
        runs["Dense (MahaBERT-SBERT)"] = results
        runs["Dense (MahaBERT-SBERT)::time"] = time.time() - t0

    hybrid = HybridRetriever(bm25=built[True], dense=dense)
    results = []
    for qi in queries:
        hits = hybrid.search(corpus[qi]["title"], top_k=top_k)
        rank = next((h.rank for h in hits if h.doc_id == qi), None)
        results.append(rank)
    runs["Hybrid (RRF)"] = results

    # ---- report ----
    header = f" {'system':>26s} {'R@1':>7s} {'R@5':>7s} {'R@10':>7s} {'MRR@10':>8s}"
    print(header)
    print(" " + "-" * (len(header) - 2))
    table = {}
    for name, ranks in runs.items():
        if "::" in name:
            continue
        found = [r for r in ranks if r is not None]
        r1 = sum(1 for r in found if r < 1) / len(ranks)
        r5 = sum(1 for r in found if r < 5) / len(ranks)
```

```

r10 = len(found) / len(ranks)
mrr = sum(1 / (r + 1) for r in found) / len(ranks)
table[name] = {"R@1": r1, "R@5": r5, "R@10": r10, "MRR@10": mrr}
print(f" {name:26s} {r1:7.3f} {r5:7.3f} {r10:7.3f} {mrr:8.3f}")

plain = table.get("BM25 (plain tokens)", {}).get("MRR@10", 0)
morph = table.get("BM25 + morphology", {}).get("MRR@10", 0)
if plain:
    print(f"\n morphology lifts BM25 MRR@10 by {(morph - plain) / plain:+.1%}")
v_plain = runs.get("BM25 (plain tokens)::vocab", 0)
v_morph = runs.get("BM25 + morphology::vocab", 0)
if v_plain:
    print(f" vocabulary: {v_plain:,} -> {v_morph:,} types "
          f"({1 - v_morph / v_plain:.1%} smaller)")
return table
# ...

```

tests/test_pipeline.py

1 of its definitions

One of the five test classes: the analyser's rules, including the guarantee that every inflected form of a word

reduces to a single root.

```
# ...
class TestMorphology:
    def test_akshara_length_ignores_matras(self):
        # घरा is 3 code points but 2 aksharas (घ + रा with a vowel sign).
        assert len("घरा") == 3
        assert akshara_len("घरा") == 2
        assert akshara_len("घर") == 2

    @pytest.mark.parametrize("surface,expected", [
        ("घरात", "घर"), ("घरातूत", "घर"), ("घरांमधूत", "घर"),
        ("घराच्या", "घर"), ("घरासाठी", "घर"),
        ("मुंबईपासूत", "मुंबई"), ("बोलूत", "बोल"),
    ])
    def test_known_analyses(self, surface, expected):
        assert stem(surface) == expected

    @pytest.mark.parametrize("group", [
        ["घर", "घरात", "घरातूत", "घरांमधूत", "घराच्या", "घरासाठी", "घरे"],
        ["शाळा", "शाळेत", "शाळेतूत", "शाळांना"],
        ["नदी", "नद्या", "नदीत"],
        ["सरकार", "सरकारने", "सरकारच्या", "सरकारला"],
        ["मुंबई", "मुंबईत", "मुंबईतील", "मुंबईपासूत"],
    ])
    def test_inflectional_family_shares_one_root(self, group):
        """The whole point of the analyser: one lemma, one index term."""
        assert len({stem(w) for w in group}) == 1

    def test_no_overstripping_of_lexical_suffixes(self):
        # कर is a real derivational suffix, but सरकार is not सर + कर.
        assert stem("सरकारने") == "सरकार"

    def test_stem_never_empties_a_word(self):
        for w in ["ला", "ने", "त", "च", "आहे", "घर", "अ"]:
            assert stem(w), f"{w} stemmed to empty string"

    def test_features_are_reported(self):
        a = analyse("घरांमधूत")
        assert isinstance(a, Analysis)
        assert a.is_inflected
        assert "ABL" in a.features

    def test_uninflected_word_is_unchanged(self):
        a = analyse("घर")
        assert a.root == "घर" and not a.is_inflected

    def test_tokenizer_handles_mixed_scripts(self):
        toks = tokenize("भारत India 2024 मध्ये")
        assert toks == ["भारत", "India", "2024", "मध्ये"]

    def test_sentence_split_on_danda(self):
        assert len(sent_tokenize("हे पहिले वाक्य आहे। हे दुसरे वाक्य आहे।")) == 2

    def test_stopwords_removed(self):
        assert "आणि" not in normalize_tokens("शाळा आणि महाविद्यालय")

    def test_morph_can_be_disabled(self):
        """The ablation switch must actually change the output."""
        assert normalize_tokens("घरात", use_morph=False) == ["घरात"]
        assert normalize_tokens("घरात", use_morph=True) == ["घर"]

    def test_empty_and_none_input(self):
        assert tokenize("") == [] and tokenize(None) == []
        assert normalize_tokens("") == []

# ...
```

The browser demo

src/lite.py

3 of its definitions

Small linear models trained to imitate the MahaBERT topic, sentiment and NER models, so the demo page can analyse pasted text with no Python. The features below are ported to JavaScript in app/demo_engine.js and checked against this Python on 1,000 articles.

```
# ...
def sentiment_features(sentence: str) -> List[str]:
    toks = tokenize(sentence)
    feats = []
    for t in toks:
        feats.append("w:" + t)
        r = stem(t)
        if r != t:
            feats.append("r:" + r)
    for a, b in zip(toks, toks[1:]):
        feats.append("b:" + a + "_" + b)
    return feats

# ...
def ner_features(toks: List[str], i: int, prev: str, prev2: str) -> List[str]:
    """Features for word i; prev/prev2 are the labels already assigned."""
    w = toks[i]
    r = stem(w)
    p = toks[i - 1] if i > 0 else "<s>"
    n = toks[i + 1] if i + 1 < len(toks) else "</s>"
    n2 = toks[i + 2] if i + 2 < len(toks) else "</s>"
    feats = [
        "bias", "w:" + w, "r:" + r, "s1:" + w[-1:], "s2:" + w[-2:], "s3:" + w[-3:],
        "p3:" + w[:3], "sh:" + _shape(w), "sw:" + str(w in STOPWORDS),
        "pw:" + p, "pr:" + stem(p), "nw:" + n, "nr:" + stem(n), "nw2:" + n2,
        "pl:" + prev, "pl2:" + prev + "|" + prev2, "plw:" + prev + "|" + r,
        "pwn:" + p + "|" + w, "wn:" + w + "|" + n,
    ]
    if i == 0:
        feats.append("first")
    return feats

# ...
def analyse(model: "Lite", title: str, text: str) -> dict:
    """The whole in-browser pipeline, mirroring src/pipeline.process with the ..."""
    from src.summarize import summarize
    title, text = (title or "").strip(), (text or "").strip()
    summ = summarize(text, k=3)
    summary = summ.text
    ent_source = f"{title}. {summary}".strip()
    sent = model.document_sentiment(summary or text)
    return {
        "topic_scores": model.topic(title, text),
        "summary": summary,
        "entities": group_entities(ent_source, model.entities(ent_source)),
        "sentiment": sent["label"],
        "polarity": sent["polarity"],
        "sentiment_distribution": sent["distribution"],
    }

# ...
```

app/server.py

2 of its definitions

Optional local server: when it runs, the demo page sends pasted text to the full models instead of the students

above.

```
# ...
def analyse(payload: dict) -> dict:
    from src.pipeline import process
    title = str(payload.get("title") or "")[:1000]
    text = str(payload.get("text") or "")[:MAX_CHARS]
    if not text.strip():
        raise ValueError("text is empty")
    with _LOCK:
        doc = process(title, text, translate=bool(payload.get("translate")))
    return doc.to_dict()

# ...
def translate(payload: dict) -> dict:
    from src import translate as mt
    texts = [str(t)[:2000] for t in (payload.get("texts") or [])][:40]
    if not texts:
        return {"translations": []}
    with _LOCK:
        return {"translations": mt.translate(texts)}

# ...
```

Marathi News Intelligence Pipeline · Source code